

Formalizing Probability Theory using the Lean Interactive Theorem Prover

Peter Pfaffelhuber

Frankfurt, May 13th, 2026

Some LEAN pioneers



Leonardo di Moura
started Lean in 2013
at Microsoft.



Kevin Buzzard
promotes Lean for
mathematicians since
2018.



Johan Commelin
led the Liquid Tensor
Experiment.



Terrence Tao
leads collaborative
formalization efforts.

- ▶ Lean is a functional programming language.
- ▶ Developed by the Lean FRO (≈ 20 people).
- ▶ Key features: meta-programming, interactive theorem proving.
- ▶ The largest **mathematical library** is mathlib, with about $2 \cdot 10^6$ lines of code.
- ▶ The graphic shows the import structure of mathlib.



- ▶ Divisibility
- ▶ Unintuitive behavior

- ▶ Every *term* has a *type*, e.g.:
 $n : \mathbb{N}$ or $\alpha : \mathbf{Type}$ or
 $\text{negneg_of_classical } (p : \mathbf{Prop}) : (p \vee \neg p) \rightarrow (p \leftrightarrow \neg \neg p)$
- ▶ There is a hierarchy of types: $\mathbf{Type} 0$ has type $\mathbf{Type} 1$ (in order to avoid logical pitfalls).
- ▶ Every type can be constructed, e.g.:
 $\mathbf{inductive} \ \mathbb{N}$ where
 $| \text{ zero} : \mathbb{N}$
 $| \text{ succ } (n : \mathbb{N}) : \mathbb{N}$
 The construction of a $\mathbf{Prop}$ -type is its proof.

- Lean uses *dependent types*. Example:

```
def Vec ( $\alpha$  : Type) ( $n$  :  $\mathbb{N}$ ) := { v : List  $\alpha$  // v.length = n }
```

Type Vec, which depends on the term $n : \mathbb{N}$

Errors in the length of a Vec are discovered at compile-time, not at run-time.

Successful compilation of a lean file



all proofs in that file are correct.

- The Curry-Howard isomorphism is described as *proofs-as-programs* and *propositions-as-types*. Example:

```
 $\forall$  x : Type*, x = x := fun x => rfl
```

Discrete Probability

```
structure DiscreteMeasure ( $\alpha$  : Type*) where  
  weight :  $\alpha \rightarrow \mathbb{R}_{\geq 0 \infty}$ 
```

```
noncomputable def toMeasure [MeasurableSpace  $\alpha$ ] :  
  DiscreteMeasure  $\alpha \rightarrow$  Measure  $\alpha$  :=  
  fun  $\mu \mapsto$  Measure.sum (fun a  $\mapsto$   $\mu$ .weight a • Measure.dirac a)
```

```
noncomputable def coin (p : unitInterval) :  
  DiscreteMeasure Bool where  
  weight  
  | true  =>  $\uparrow p$   
  | false =>  $\uparrow (1 - p)$ 
```

Discrete Probability

```
noncomputable def binom (p : unitInterval) :
```

```
   $\mathbb{N} \rightarrow$  DiscreteMeasure  $\mathbb{N}$ 
```

```
| 0 => dirac 0
```

```
| n + 1 => do
```

```
  let X  $\leftarrow$  coin p
```

```
  let Y  $\leftarrow$  binom p n
```

```
  return ( $\uparrow$ X + Y)
```

```
theorem binom_eq_count_true (p : unitInterval) (n :  $\mathbb{N}$ ) :
```

```
  binom p n = do
```

```
    let X  $\leftarrow$  iidSequence n (coin p)
```

```
    return X.count true
```


Random Variables – without Basic Space

GÖTZ KERSTING *

June 11, 2007

HERMANN DINGES

VARIABLES, IN PARTICULAR RANDOM VARIABLES

The probability monad

- ▶ $\text{DiscreteMeasure } \alpha$ = discrete probability measures on α .
- ▶ $\text{dirac}_\alpha : \alpha \rightarrow \text{DiscreteMeasure } \alpha, \quad x \mapsto \delta_x$.
- ▶ A *Markov kernel* $\kappa : \alpha \rightarrow \text{DiscreteMeasure } \beta$ extends to

$$\text{bind } \kappa : \text{DiscreteMeasure } \alpha \rightarrow \text{DiscreteMeasure } \beta.$$

For $\kappa : \alpha \rightarrow \text{DiscreteMeasure } \beta$, $\eta : \beta \rightarrow \text{DiscreteMeasure } \gamma$:

$$\text{bind } (\text{dirac}_\alpha) \mu = \mu$$

$$\text{bind } \kappa (\delta_x) = \kappa(x)$$

$$\text{bind } (\text{bind } \eta \circ \kappa) = \text{bind } \eta \circ \text{bind } \kappa \quad (\text{Chapman--Kolmogorov})$$

Pushforward and mixture are derived from dirac and bind :

$$\text{map } f = \text{bind } (\text{dirac} \circ f) \quad (\text{for } f : \alpha \rightarrow \beta; \text{ law of } f(X) \text{ for } X \sim \mu)$$

$$\text{join} = \text{bind id} \quad (\text{average over a random measure } \Xi \sim m)$$

The probability monad

- ▶ $\text{dirac } a = \text{Dirac measure } \delta_a$.
- ▶ $\mu.\text{map } f = \text{pushforward}$: law of $f(X)$ when $X \sim \mu$.
- ▶ $\Xi.\text{join} = \text{mixture}$: average over a random measure $\mu \sim \Xi$.
- ▶ $\mu.\text{bind } f = \text{sample } X \sim \mu, \text{ then } Y \sim f(X)$.

```
noncomputable def map (g :  $\alpha \rightarrow \beta$ ) ( $\mu$  : DiscreteMeasure  $\alpha$ ) :  
  DiscreteMeasure  $\beta$  :=  
   $\langle \text{fun } b \mapsto \sum' a, (g^{-1} \{b\}).\text{indicator } \mu.\text{weight } a \rangle$ 
```

```
noncomputable def join ( $\Xi$  : DiscreteMeasure (DiscreteMeasure  $\alpha$ )) :  
  DiscreteMeasure  $\alpha$  :=  
   $\langle \text{fun } a \mapsto \sum' \mu, \Xi \mu * \mu a \rangle$ 
```

```
noncomputable def bind ( $\mu$  : DiscreteMeasure  $\alpha$ )  
  (f :  $\alpha \rightarrow \text{DiscreteMeasure } \beta$ ) : DiscreteMeasure  $\beta$  :=  
  ( $\mu.\text{map } f$ ).join
```

The probability monad

These operations satisfy the monad laws, the most involved being associativity of bind:

```
theorem bind_bind ( $\mu$  : DiscreteMeasure  $\alpha$ )  
  ( $f$  :  $\alpha \rightarrow$  DiscreteMeasure  $\beta$ ) ( $g$  :  $\beta \rightarrow$  DiscreteMeasure  $\gamma$ ) :  
  ( $\mu$ .bind  $f$ ).bind  $g$  =  $\mu$ .bind (fun  $a \mapsto (f\ a)$ ).bind  $g$ )
```

```
instance : LawfulMonad DiscreteMeasure
```

This unlocks **do**-notation, which desugars to nested binds:

```
do  
  let  $X \leftarrow \mu$   
  let  $Y \leftarrow \nu$   
  return ( $f\ X\ Y$ )  
-- is the same as  
 $\mu$ .bind (fun  $X \mapsto \nu$ .bind (fun  $Y \mapsto \text{dirac } (f\ X\ Y)$ )))
```

Discrete Probability (Insights from formalization)

- ▶ Discrete probability theory is like probability theory without any measurability issues.
(This makes discrete probability measures a lawful monad.)
- ▶ Using **do**-Notation requires use of monads, and make definitions more readable.
- ▶ The binomial distribution can be defined in several equivalent ways.
- ▶ Formalizing basic mathematics already feels like developing a theory.
- ▶ Mathematically easy things can be hard to prove formally.
(Equivalence of all definitions of the binomial distribution.)

Formalization of Brownian motion in Lean

Rémy Degenne, David Ledvinka, Etienne Marion, Peter Pfaffelhuber

Abstract. Brownian motion is a building block in modern probability theory. In this paper, we describe a formalization of Brownian motion using the LEAN theorem prover. We build on the existing measure-theoretic foundations in Lean's mathematical library, Mathlib, and we develop several key components needed for the construction of Brownian motion, including the Carathéodory and Kolmogorov extension theorems, Gaussian measures in Banach spaces, and the Kolmogorov-Chentsov theorem for path continuity.